

Restaurant Ops Hub: System Architecture

One portal for every team and every vendor, with an AI agent doing the follow-ups

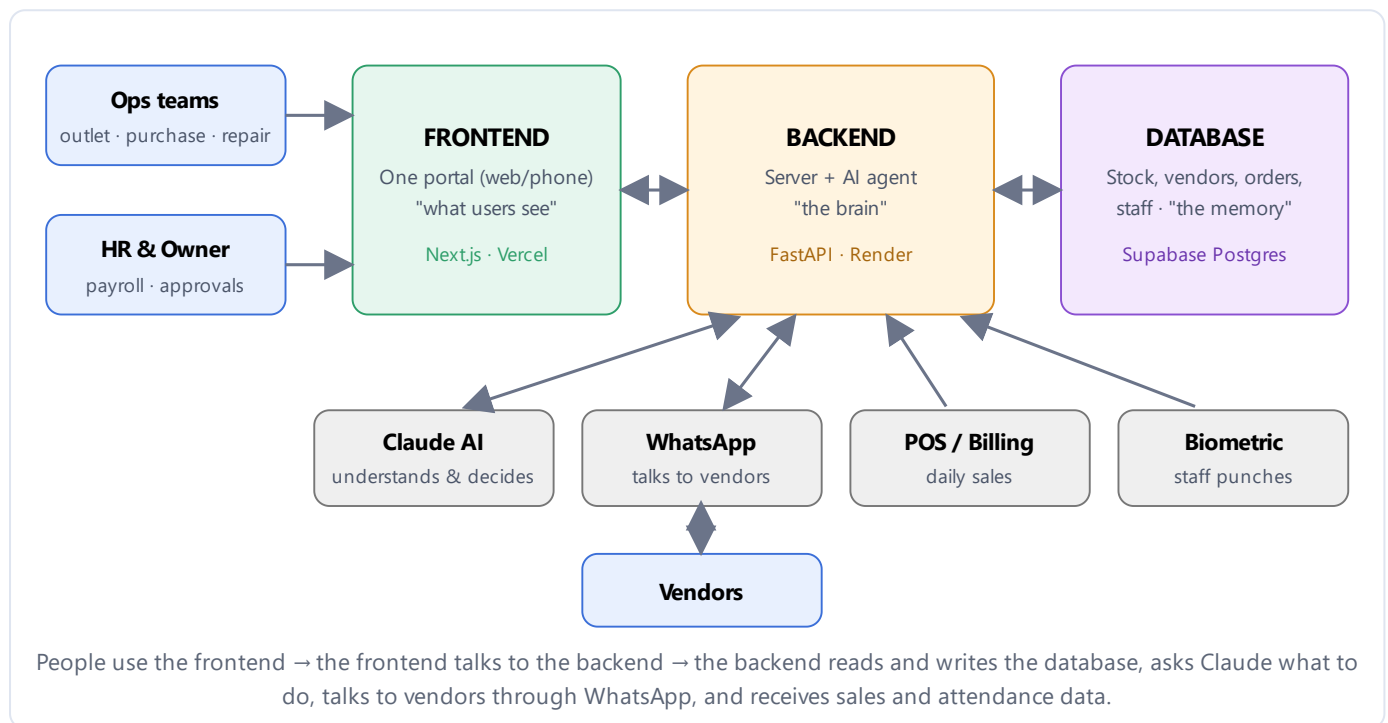
Samiya Sehgal

URN: 2302662

1. What the system does (in one breath)

A restaurant chain has **many teams** (outlets, purchase, maintenance, HR, owner) and **many vendors** (raw material, packaging, gas, electricity, IT, hiring, utensils). This system puts **all of them in one portal**. An **AI agent** predicts shortages from sales, orders, reminds and books vendors on **WhatsApp** (auto-orders under a ₹ limit, asks approval above it), and turns **biometric attendance** into **payroll**.

2. Simple architecture (the big picture)



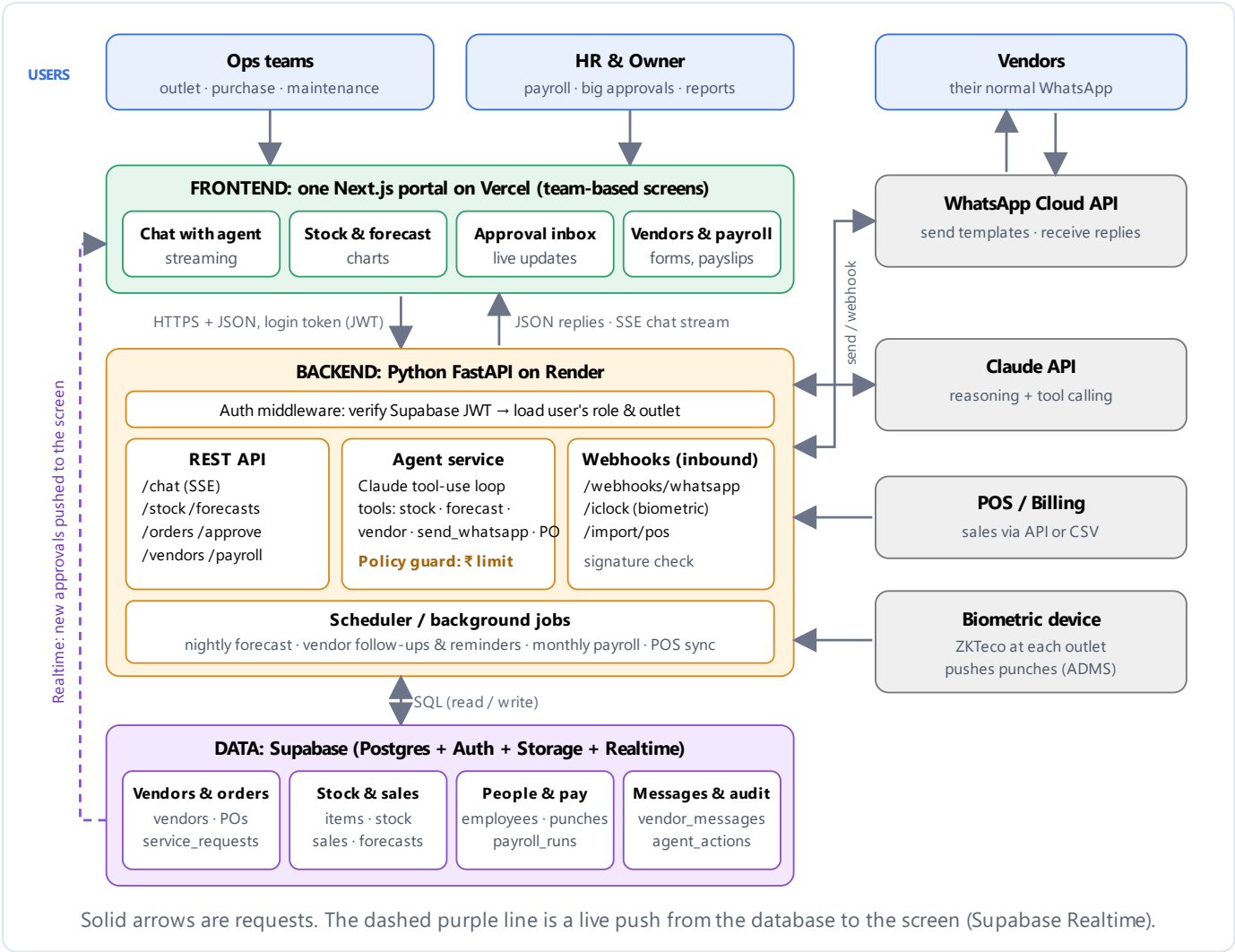
3. How it works, in plain words

Part	Think of it as...	What it does here
Frontend	The restaurant's front counter	The screens the manager and HR use: a chat box, stock and forecast charts, an approval inbox and payroll pages. It shows things and collects clicks. It makes no decisions.
Backend	The kitchen and the manager's brain	Receives every request, checks who is asking, runs the AI agent, enforces the ₹ auto-order limit, sends WhatsApp messages and runs the nightly and monthly jobs.
Database	The register / ledger	Stores vendors, items, stock, sales, orders, every vendor message, employees, attendance punches and payroll runs.

How the frontend and backend connect: the frontend never talks to WhatsApp, Claude or the database directly. It sends a request to the backend over the internet (**HTTPS**) carrying the user's login token. The backend checks the token, does the work and sends back an answer in **JSON** (a simple text format both sides understand). For chat, the answer is **streamed** word by word so the manager sees the agent "typing".

4. Detailed architecture

The same system, one level deeper: what lives inside each layer and **what kind of connection** joins them.



People Frontend Backend Data Outside services

Summary: a 3-layer web app (Next.js → FastAPI → Supabase) with an **AI agent inside the backend**. Claude decides, the backend's tools act, and a policy guard keeps money decisions safe. WhatsApp, POS and biometric data come in through webhooks and scheduled jobs.

Teams & vendors in one portal

Team	What they do in the portal	Vendors they deal with
Outlet managers	Check stock, raise requests, chat with the agent	Raw material (veg, fruits, maida, dairy)
Purchase / store	Vendor list, rates, purchase orders, godown transfers	Raw material, packaging, utensils
Maintenance	Repair tickets, slot bookings, follow-ups	Gas, electricity, equipment repair, IT
HR	Attendance, payroll, hiring requests	Hiring and chef agencies
Owner	Approves big orders, sets ₹ limits, sees all reports	All vendors (overview)

5. Deep dive: frontend, backend and how they connect

Frontend (Next.js on Vercel)

- **Login:** Supabase Auth (email or phone OTP). The browser keeps a short-lived **JWT token** that proves who the user is.
- **Chat screen:** sends the manager's message to `POST /chat` and shows the reply as it streams in (Server-Sent Events).
- **Dashboards:** stock "days left", forecasts and open orders, fetched with `GET` calls and drawn with Recharts.
- **Approval inbox:** orders above the ₹ limit show up **instantly** through Supabase Realtime, with Approve / Reject buttons.
- **One portal, team-based screens:** purchase sees orders and vendors, maintenance sees repair tickets, HR sees payroll, outlet managers see their outlet, the owner sees everything.
- Installable as a **PWA** so it feels like an app on the manager's phone.

Backend (FastAPI on Render)

- **Auth middleware:** every request's JWT is verified, then the user's role and outlet are loaded.
- **REST API:** plain endpoints for stock, vendors, orders, approvals and payroll.
- **Agent service:** sends the chat plus a list of **tools** to Claude. Claude replies `"call check_stock(maida)"`, the backend runs it, returns the result, and the loop repeats until Claude gives a final answer.
- **Policy guard:** sits between the agent and the real world. A PO under the limit to an approved vendor is sent. Anything else becomes a "pending approval".
- **Webhooks:** receive WhatsApp replies and biometric punches, after checking their signatures.
- **Scheduler:** nightly forecast, follow-ups, monthly payroll.

How the frontend and backend talk: the 5 connection types

Connection	Used for	How it works (simple → technical)
1. REST (HTTPS + JSON)	Loading stock, saving a vendor, approving an order	The frontend asks a question and the backend answers once. <code>GET /stock?outlet=2 → {"maida": {"qty": 18, "days_left": 4}}</code> . The JWT goes in the <code>Authorization</code> header.
2. Streaming (SSE)	Agent chat	One request, many small replies. The connection stays open while the agent thinks and calls tools, so text appears live: "Checking stock... Maida runs out in 4 days..."
3. Realtime push	Approval inbox, order status	The frontend subscribes to the <code>purchase_orders</code> table. When the backend inserts a "pending" row, Supabase pushes it to the screen with no refresh.
4. Webhooks (inbound)	Vendor replies, biometric punches	Outside systems call <i>us</i> . WhatsApp POSTs each vendor reply to <code>/webhooks/whatsapp</code> . The backend matches it to the open order and updates it.
5. Scheduled jobs	Forecasts, reminders, payroll	Nobody clicks anything. A timer (cron) wakes the backend: every night at 2 am for forecasting, every 15 min for follow-ups, on the 1st of the month for payroll.

Inside the agent: one turn, step by step

1. Manager: "Do we have enough maida for the weekend?" → frontend → `POST /chat`.
2. Backend sends Claude the message, the restaurant context and the tool list.
3. Claude asks for `check_stock("maida")` then `forecast_item("maida", days=3)`. The backend runs both against the database.
4. Claude proposes: shift 20 kg from the godown **or** order 50 kg (₹1,900) from Sharma Traders.
5. The manager picks "order". The agent calls `create_po` → the **policy guard** checks ₹1,900 < ₹5,000 limit and that the vendor is approved → it calls `send_whatsapp`. Every step is saved in `agent_actions` as an audit trail.

6. Key flows end to end

Flow	Path through the system
Urgent repair ("cylinder leaking")	Chat → /chat → agent → find_vendor(category=gas, outlet) → send_whatsapp (template: "urgent, reply with a time") → service_requests row = <i>waiting</i> → vendor replies "YES 3 pm" → webhook → status = <i>booked</i> , slot saved → Realtime update on the manager's screen → scheduler sends a reminder at 2:30 pm. No reply in 20 min → backup vendor is messaged.
Shortage → auto order	POS sync loads sales → nightly job computes usage × recent trend → maida "days left" = 4 (below the 5-day threshold) → checks godown first → if an order is needed and it's under the ₹ limit → PO sent on WhatsApp automatically. Otherwise → pending approval → Realtime → manager taps Approve → PO sent.
Monthly payroll	Biometric device pushes punches all month → attendance_punches → on the 1st, the payroll job turns punches into daily status (present / half day / late / absent) using the shift rules → salary minus deductions plus overtime → payroll_runs (draft) → HR reviews only the flagged rows → approve → payslip PDFs go to Storage.

7. Data model (main tables)

Group	Tables	Why it matters
Vendors & orders	vendors, vendor_categories, purchase_orders (status, amount, auto/approved), service_requests	Who to call for what, and the state of every order or repair
Stock & sales	outlets, items, stock_levels, godown_transfers, sales, forecasts	Input for shortage prediction
People & pay	users (role), employees, shifts, attendance_punches, payroll_runs	Automatic, explainable salary
Messages & audit	vendor_messages, agent_actions, settings (₹ limit per outlet)	Proof of every message and AI decision

8. Security & reliability

- **Login + roles:** JWT on every request. Row-Level Security so an outlet manager only sees their outlet.
 - **AI guardrails:** the agent acts only through tools, and money actions pass the policy guard.
 - **Webhook safety:** signatures verified, and only registered biometric devices are accepted.
- **No double orders:** every PO has an idempotency key, so a retry never orders twice.
 - **Retries:** failed WhatsApp sends retry with back-off, then alert the manager.
 - **Secrets:** API keys stay on the server, never in the frontend.

9. Hosting & free tools

Layer	Tool	Free tier / note
Frontend	Next.js + Recharts on Vercel	Hobby plan free
Backend + jobs	FastAPI on Render + Supabase Cron	Free tier sleeps when idle. Use a paid instance in production.
Data & auth	Supabase (DB, Auth, Realtime)	500 MB DB free
AI	Claude API (tool use)	Starter credits, then pay-as-you-go
Messaging	WhatsApp Cloud API (Meta)	Free test number. Small charge per business-started message.